

TRABAJO DE FIN DE GRADO

Desarrollo de Aplicaciones Multiplataforma

GestCar

Aplicación Android para la gestión de mantenimiento y costes vehiculares

ENTREGA PARCIAL 1: DISEÑO Y ANÁLISIS

Autor: Diego Buiza Rivas

Centro: IES Isidra de Guzmán

Ciclo: DAM – Desarrollo de Aplicaciones Multiplataforma

Curso académico: 2025–2026

Índice

Índice.....	2
1. Introducción.....	4
1.1. Contexto y motivación.....	4
1.2. Objetivos del documento.....	4
1.3. Alcance del proyecto.....	4
2. Análisis de requisitos.....	5
2.1. Requisitos funcionales.....	5
2.1.1. Gestión de vehículos.....	5
2.1.2. Gestión de repostajes.....	5
2.1.3. Gestión de mantenimientos y reparaciones.....	5
2.1.4. Gestión de gastos periódicos.....	6
2.1.5. Recordatorios.....	6
2.1.6. Estadísticas y cálculos.....	7
2.2. Requisitos no funcionales.....	7
3. Análisis de casos de uso.....	9
3.1. Actores del sistema.....	9
3.2. Diagrama general de casos de uso.....	9
3.3. Descripción detallada de casos de uso.....	9
CU-01: Registrar vehículo.....	9
CU-02: Registrar repostaje.....	10
CU-03: Registrar mantenimiento.....	10
CU-04: Crear recordatorio.....	10
CU-05: Consultar estadísticas.....	11
4. Diseño del modelo de datos.....	12
4.1. Modelo entidad-relación.....	12
4.1.1. Entidades y atributos.....	12
4.1.2. Relaciones entre entidades.....	14
4.2. Esquema de la base de datos (Room / SQLite).....	14
5. Diseño de la arquitectura.....	15
5.1. Patrón arquitectónico: MVVM.....	15
5.1.1. Capa Model (Modelo).....	15
5.1.2. Capa View (Vista).....	15
5.1.3. Capa ViewModel.....	15
5.2. Diagrama de componentes.....	16
5.3. Flujo de datos.....	17
5.4. Gestión de dependencias.....	17
6. Diseño de la interfaz de usuario.....	18
6.1. Principios de diseño.....	18
6.2. Mapa de navegación.....	18
6.3. Descripción de pantallas principales.....	18
6.3.1. Pantalla principal – Listado de vehículos.....	18
6.3.2. Pantalla de detalle de vehículo.....	19
6.3.3. Formularios de registro.....	19
6.3.4. Pantalla de estadísticas.....	19
6.3.5. Pantalla de recordatorios.....	19
6.4. Paleta de colores propuesta.....	19
7. Tecnologías y herramientas.....	21
7.1. Justificación del stack tecnológico.....	21
7.2. Dependencias principales (build.gradle.kts).....	21
7.3. Entorno de desarrollo y pruebas.....	22
8. Planificación preliminar.....	23
9. Glosario de términos.....	24
10. Conclusiones de la fase de análisis y diseño.....	25

1. Introducción

1.1. Contexto y motivación

En la actualidad, la mayoría de los conductores carecen de un sistema unificado y digital que les permita registrar, consultar y analizar toda la información relevante relacionada con el uso y mantenimiento de sus vehículos. La información sobre repostajes, cambios de aceite, revisiones periódicas, pagos de seguros, inspecciones técnicas de vehículos (ITV) y demás operaciones suele quedar dispersa entre facturas en papel, notas manuales, correos electrónicos o aplicaciones aisladas de terceros (la app del seguro, la app de la gasolinera habitual, etc.). Esta fragmentación impide al usuario tener una visión global del estado real de su vehículo, dificulta la previsión de mantenimientos futuros y hace prácticamente imposible calcular con precisión el coste real por kilómetro recorrido.

GestCar nace como respuesta a esta necesidad, proponiendo una aplicación Android nativa que centralice toda esta información en un único lugar, ofreciendo al usuario un historial completo, cálculos automáticos y una interfaz intuitiva para gestionar uno o varios vehículos de forma sencilla y eficaz.

1.2. Objetivos del documento

El presente documento constituye la primera entrega parcial del Trabajo de Fin de Grado y tiene como objetivo presentar el análisis y diseño completo del sistema GestCar. Se cubren los siguientes aspectos fundamentales:

- Análisis de requisitos funcionales y no funcionales del sistema.
- Identificación y descripción detallada de los actores y casos de uso.
- Diseño del modelo de datos y esquema de base de datos relacional.
- Definición de la arquitectura de software basada en el patrón MVVM.
- Diseño de la interfaz de usuario y mapa de navegación.
- Descripción de las tecnologías seleccionadas y justificación de las decisiones técnicas.

1.3. Alcance del proyecto

GestCar está orientada inicialmente a usuarios particulares que desean llevar un control detallado de sus vehículos. No obstante, el diseño contempla la posibilidad de extensión futura a un contexto profesional o empresarial (flotas de vehículos). Los tipos de vehículos soportados inicialmente serían coches, motocicletas y furgonetas. La aplicación funcionará íntegramente en modo local (offline), almacenando toda la información en el dispositivo del usuario mediante una base de datos SQLite gestionada a través de la biblioteca Room, con posibilidad de utilizar un servicio de base de datos online en el futuro, como Firebase.

2. Análisis de requisitos

2.1. Requisitos funcionales

Los requisitos funcionales describen las capacidades que el sistema debe ofrecer al usuario. Se han organizado por áreas funcionales para facilitar su comprensión y trazabilidad con los casos de uso.

2.1.1. Gestión de vehículos

ID	Requisito	Descripción	Prioridad
RF-01	Alta de vehículo	El usuario podrá registrar un nuevo vehículo indicando marca, modelo, año, tipo (coche/moto/furgoneta), matrícula, kilometraje inicial y, opcionalmente, una imagen.	Alta
RF-02	Edición de vehículo	El usuario podrá modificar cualquier dato de un vehículo previamente registrado.	Alta
RF-03	Eliminación de vehículo	El usuario podrá eliminar un vehículo y, en cascada, todos sus registros asociados (gastos, mantenimientos, repostajes y recordatorios).	Alta
RF-04	Listado de vehículos	La aplicación mostrará una lista de todos los vehículos registrados con información resumida (marca, modelo, matrícula y kilometraje actual).	Alta
RF-05	Ficha de vehículo	El usuario podrá acceder a una vista de detalle con toda la información del vehículo, incluyendo estadísticas calculadas.	Media

2.1.2. Gestión de repostajes

ID	Requisito	Descripción	Prioridad
RF-06	Registro de repostaje	El usuario podrá registrar un repostaje indicando fecha, kilómetros actuales, litros, precio por litro, importe total y tipo de combustible. El sistema calculará automáticamente el importe total si se proporcionan litros y precio/litro.	Alta
RF-07	Edición de repostaje	El usuario podrá modificar los datos de un repostaje previamente registrado.	Alta
RF-08	Eliminación de repostaje	El usuario podrá eliminar un repostaje del historial.	Alta
RF-09	Historial de repostajes	La aplicación mostrará el listado cronológico de repostajes de un vehículo con opciones de filtrado por rango de fechas.	Alta
RF-10	Cálculo de consumo medio	El sistema calculará automáticamente el consumo medio (L/100km) a partir de los datos de repostaje registrados.	Alta

2.1.3. Gestión de mantenimientos y reparaciones

ID	Requisito	Descripción	Prioridad
RF-11	Registro de mantenimiento	El usuario podrá registrar una operación de mantenimiento indicando tipo (aceite, neumáticos, frenos, filtros, etc.), fecha, kilometraje, coste, taller y observaciones.	Alta
RF-12	Registro de reparación	El usuario podrá registrar reparaciones imprevistas con tipo de avería, fecha, kilometraje, coste, taller y descripción detallada.	Alta
RF-13	Edición de mantenimiento/reparación	El usuario podrá editar los datos de cualquier operación registrada.	Media
RF-14	Eliminación de mantenimiento/reparación	El usuario podrá eliminar registros de mantenimiento o reparación.	Media
RF-15	Historial de operaciones	La aplicación mostrará un listado cronológico de todas las operaciones de mantenimiento y reparación de un vehículo.	Alta

2.1.4. Gestión de gastos periódicos

ID	Requisito	Descripción	Prioridad
RF-16	Registro de gasto periódico	El usuario podrá registrar gastos recurrentes como seguro, ITV, impuesto de circulación, aparcamiento, etc., indicando concepto, importe, fecha, periodicidad y fecha de vencimiento.	Alta
RF-17	Edición de gasto periódico	El usuario podrá modificar los datos de un gasto periódico registrado.	Media
RF-18	Eliminación de gasto periódico	El usuario podrá eliminar un gasto periódico del sistema.	Media
RF-19	Listado de gastos periódicos	La aplicación mostrará los gastos periódicos asociados a cada vehículo.	Alta

2.1.5. Recordatorios

ID	Requisito	Descripción	Prioridad
RF-20	Creación de recordatorio	El usuario podrá crear recordatorios asociados a un vehículo (próxima ITV, cambio de aceite, renovación de seguro, etc.) indicando concepto, fecha límite y, opcionalmente, kilometraje límite.	Alta
RF-21	Notificación de recordatorio	El sistema mostrará un indicador visual o alerta cuando un recordatorio esté próximo a su fecha de vencimiento o se haya superado el kilometraje límite.	Media
RF-22	Edición de recordatorio	El usuario podrá modificar los datos de un recordatorio.	Media
RF-23	Eliminación de recordatorio	El usuario podrá eliminar recordatorios.	Media
RF-24	Marcar recordatorio como completado	El usuario podrá marcar un recordatorio como realizado, registrándose la fecha de cumplimiento.	Media

2.1.6. Estadísticas y cálculos

ID	Requisito	Descripción	Prioridad
RF-25	Coste por kilómetro	El sistema calculará el coste total por kilómetro recorrido sumando todos los gastos (repostajes, mantenimientos, reparaciones y gastos periódicos) y dividiéndolo entre los kilómetros recorridos.	Alta
RF-26	Resumen de gastos	La aplicación mostrará un resumen de gastos por categoría (combustible, mantenimiento, reparaciones, gastos periódicos) y por período temporal (mes, trimestre, año).	Alta
RF-27	Gráficos de evolución	El sistema mostrará gráficos básicos (barras o líneas) que representen la evolución del consumo y los gastos a lo largo del tiempo.	Baja
RF-28	Exportación de datos	El usuario podrá exportar el historial de gastos de un vehículo en formato CSV para su análisis externo.	Baja

2.2. Requisitos no funcionales

Los requisitos no funcionales definen las restricciones y cualidades del sistema que no están directamente relacionadas con funcionalidades específicas, sino con aspectos transversales como rendimiento, usabilidad, seguridad y mantenibilidad.

ID	Categoría	Descripción
RNF-01	Rendimiento	La aplicación deberá responder a las interacciones del usuario en menos de 2 segundos para operaciones comunes (consultas, listados). Las operaciones de cálculo podrán tomar hasta 5 segundos.
RNF-02	Usabilidad	La interfaz seguirá las guías de diseño Material Design 3 de Google, garantizando coherencia visual y facilidad de aprendizaje. El usuario deberá poder completar cualquier operación CRUD en un máximo de 3 pasos.
RNF-03	Compatibilidad	La aplicación será compatible con dispositivos Android con API 26 (Android 8.0 Oreo) o superior, cubriendo más del 95% del parque de dispositivos activo.
RNF-04	Almacenamiento local	Todos los datos se almacenarán localmente en el dispositivo mediante SQLite (Room), sin requerir conexión a internet para su funcionamiento básico.
RNF-05	Mantenibilidad	El código seguirá la arquitectura MVVM con separación clara de responsabilidades, facilitando la extensión y el mantenimiento futuro del proyecto.
RNF-06	Escalabilidad	El diseño de la base de datos y la arquitectura permitirán añadir nuevas funcionalidades (sincronización en la nube, gestión de flotas) sin necesidad de reestructurar el sistema existente.
RNF-07	Integridad de datos	El sistema garantizará la consistencia de los datos mediante el uso de claves foráneas, restricciones de unicidad y eliminación en cascada cuando corresponda.
RNF-08	Accesibilidad	La interfaz incluirá descripciones de contenido (contentDescription) en los elementos visuales para compatibilidad con lectores de pantalla (TalkBack).
RNF-09	Idioma	La aplicación se desarrollará íntegramente en español, siendo el idioma principal de la interfaz y los mensajes del sistema.
RNF-10	Tamaño de la aplicación	El APK final no deberá superar los 30 MB de tamaño para facilitar su distribución e instalación.

3. Análisis de casos de uso

3.1. Actores del sistema

El sistema GestCar contempla un único actor principal:

- **Usuario:** Persona propietaria o responsable de uno o más vehículos que utiliza la aplicación para registrar y consultar información de mantenimiento, repostajes, gastos y recordatorios.

No se contemplan roles adicionales (administrador, técnico, etc.) en esta primera versión del sistema, dado que la aplicación funciona íntegramente en modo local y no involucra interacción con otros usuarios ni con sistemas externos.

3.2. Diagrama general de casos de uso

A continuación se presenta una descripción textual del diagrama de casos de uso. Se recomienda elaborar el diagrama UML correspondiente utilizando una herramienta como StarUML, PlantUML o draw.io para la versión final de la memoria.

El actor Usuario interactúa con los siguientes paquetes de casos de uso:

- **Gestión de Vehículos:** Registrar vehículo, Editar vehículo, Eliminar vehículo, Consultar vehículos.
- **Gestión de Repostajes:** Registrar repostaje, Editar repostaje, Eliminar repostaje, Consultar historial de repostajes.
- **Gestión de Mantenimientos:** Registrar mantenimiento/repación, Editar mantenimiento, Eliminar mantenimiento, Consultar historial.
- **Gestión de Gastos Periódicos:** Registrar gasto, Editar gasto, Eliminar gasto, Consultar gastos.
- **Recordatorios:** Crear recordatorio, Editar recordatorio, Eliminar recordatorio, Marcar como completado.
- **Estadísticas:** Consultar consumo medio, Consultar coste/km, Consultar resumen de gastos, Exportar datos.

3.3. Descripción detallada de casos de uso

CU-01: Registrar vehículo

Campo	Descripción
Actor principal	Usuario
Precondiciones	La aplicación está abierta y el usuario se encuentra en la pantalla principal o en el listado de vehículos.
Flujo principal	1. El usuario pulsa el botón «Añadir vehículo». 2. El sistema muestra el formulario de alta con los campos: marca, modelo, año, tipo, matrícula, kilometraje inicial e imagen (opcional). 3. El usuario cumplimenta los campos obligatorios y pulsa «Guardar». 4. El sistema valida los datos introducidos. 5. El sistema almacena el vehículo en la base de datos. 6. El sistema muestra un mensaje de confirmación y redirige al listado actualizado.
Flujos alternativos	4a. Si los datos no son válidos (campos obligatorios vacíos, año fuera de rango, matrícula duplicada), el sistema muestra un mensaje de error indicando los campos incorrectos y el usuario corrige los datos.
Postcondiciones	El vehículo queda registrado en el sistema y aparece en el listado.
Requisitos relacionados	RF-01, RF-04

CU-02: Registrar repostaje

Campo	Descripción
Actor principal	Usuario
Precondiciones	Existe al menos un vehículo registrado en el sistema. El usuario ha seleccionado un vehículo.
Flujo principal	1. El usuario accede a la sección de repostajes del vehículo seleccionado. 2. El usuario pulsa «Nuevo repostaje». 3. El sistema muestra el formulario con los campos: fecha (por defecto hoy), kilómetros actuales, litros, precio/litro, tipo de combustible. 4. El usuario introduce los datos y pulsa «Guardar». 5. El sistema calcula automáticamente el importe total (litros × precio/litro). 6. El sistema almacena el repostaje y actualiza el kilometraje del vehículo. 7. El sistema recalcula el consumo medio y redirige al historial.
Flujos alternativos	6a. Si el kilometraje introducido es inferior al último registrado, el sistema muestra una advertencia y solicita confirmación al usuario.
Postcondiciones	El repostaje queda registrado. El kilometraje del vehículo se actualiza. El consumo medio se recalcula.
Requisitos relacionados	RF-06, RF-09, RF-10

CU-03: Registrar mantenimiento

Campo	Descripción
Actor principal	Usuario
Precondiciones	Existe al menos un vehículo registrado. El usuario ha seleccionado un vehículo.
Flujo principal	1. El usuario accede a la sección de mantenimientos del vehículo. 2. Pulsa «Nuevo mantenimiento». 3. El sistema muestra el formulario: tipo de operación (lista predefinida + personalizada), categoría (mantenimiento/reparación), fecha, kilometraje, coste, taller (opcional), observaciones. 4. El usuario completa los datos y pulsa «Guardar». 5. El sistema almacena la operación y actualiza el kilometraje si procede. 6. Se muestra confirmación y se redirige al historial.
Flujos alternativos	3a. Si el tipo de operación seleccionado tiene un recordatorio activo asociado, el sistema ofrece marcarlo como completado automáticamente.
Postcondiciones	El mantenimiento queda registrado. El kilometraje se actualiza si el valor introducido es superior al actual.
Requisitos relacionados	RF-11, RF-12, RF-15, RF-24

CU-04: Crear recordatorio

Campo	Descripción
Actor principal	Usuario
Precondiciones	Existe al menos un vehículo registrado.
Flujo principal	1. El usuario accede a la sección de recordatorios del vehículo. 2. Pulsa «Nuevo recordatorio». 3. El sistema muestra el formulario: concepto, fecha límite, kilometraje límite (opcional), notas. 4. El usuario completa los datos y pulsa «Guardar». 5. El sistema almacena el recordatorio y programa la alerta visual. 6. El recordatorio aparece en la lista con su estado (pendiente/próximo/vencido).
Flujos alternativos	4a. Si ya existe un recordatorio activo con el mismo concepto para el mismo vehículo, el sistema advierte al usuario.
Postcondiciones	El recordatorio queda creado y se mostrará un indicador visual cuando se aproxime la fecha o el kilometraje límite.
Requisitos relacionados	RF-20, RF-21

CU-05: Consultar estadísticas

Campo	Descripción
Actor principal	Usuario
Precondiciones	Existe al menos un vehículo registrado con datos de gastos o repostajes.
Flujo principal	1. El usuario selecciona un vehículo y accede a la sección de estadísticas. 2. El sistema recopila todos los gastos registrados (repostajes, mantenimientos, reparaciones, gastos periódicos). 3. El sistema calcula y muestra: consumo medio (L/100km), coste por kilómetro, desglose de gastos por categoría, evolución temporal de gastos. 4. El usuario puede filtrar por período (mes, trimestre, año) o por categoría. 5. Opcionalmente, el usuario puede exportar los datos en formato CSV.
Flujos alternativos	2a. Si no hay suficientes datos para calcular estadísticas significativas, el sistema muestra un mensaje informativo indicando qué datos faltan.
Postcondiciones	Se muestran las estadísticas calculadas. Opcionalmente, se genera el archivo CSV de exportación.
Requisitos relacionados	RF-10, RF-25, RF-26, RF-27, RF-28

4. Diseño del modelo de datos

4.1. Modelo entidad-relación

El modelo de datos de GestCar está compuesto por cinco entidades principales y sus relaciones. A continuación se describe cada entidad, sus atributos y las relaciones entre ellas.

4.1.1. Entidades y atributos

Entidad: Vehículo

Atributo	Tipo	Restricciones	Descripción
id	Long	PK, autoincremental	Identificador único del vehículo
marca	String	NOT NULL	Marca del fabricante
modelo	String	NOT NULL	Modelo del vehículo
anio	Int	NOT NULL, >= 1900	Año de fabricación
tipo	Enum	NOT NULL	Tipo: COCHE, MOTO, FURGONETA
matricula	String	NOT NULL, UNIQUE	Matrícula del vehículo
kilometraje	Double	NOT NULL, >= 0	Kilometraje actual
tipoCombustible	Enum	NULL	Gasolina, Diésel, Eléctrico, Híbrido, GLP
imagenUri	String	NULL	URI de la imagen del vehículo
fechaAlta	Long (timestamp)	NOT NULL	Fecha de registro en la app
notas	String	NULL	Observaciones generales

Entidad: Repostaje

Atributo	Tipo	Restricciones	Descripción
id	Long	PK, autoincremental	Identificador único del repostaje
vehiculoid	Long	FK → Vehiculo, NOT NULL	Vehículo asociado
fecha	Long (timestamp)	NOT NULL	Fecha del repostaje
kilometros	Double	NOT NULL, >= 0	Kilómetros en el odómetro
litros	Double	NOT NULL, > 0	Litros repostados
precioPorLitro	Double	NOT NULL, > 0	Precio por litro en euros
importeTotal	Double	NOT NULL, > 0	Coste total calculado
llenoCompleto	Boolean	NOT NULL, default true	Indica si fue depósito lleno
gasolinera	String	NULL	Nombre de la gasolinera
notas	String	NULL	Observaciones

Entidad: Mantenimiento

Atributo	Tipo	Restricciones	Descripción
id	Long	PK, autoincremental	Identificador único
vehiculoid	Long	FK → Vehiculo, NOT NULL	Vehículo asociado
tipo	String	NOT NULL	Tipo de operación (aceite, neumáticos, frenos, filtros, embrague, etc.)
categoria	Enum	NOT NULL	MANTENIMIENTO o REPARACION
fecha	Long (timestamp)	NOT NULL	Fecha de la operación
kilometros	Double	NULL	Kilómetros en el momento
coste	Double	NOT NULL, >= 0	Coste de la operación en euros
taller	String	NULL	Nombre del taller
descripcion	String	NULL	Descripción detallada o notas

Entidad: GastoPeriodico

Atributo	Tipo	Restricciones	Descripción
id	Long	PK, autoincremental	Identificador único
vehiculoid	Long	FK → Vehiculo, NOT NULL	Vehículo asociado
concepto	String	NOT NULL	Tipo de gasto (seguro, ITV, impuesto circulación, parking, etc.)
importe	Double	NOT NULL, > 0	Importe del gasto en euros
fecha	Long (timestamp)	NOT NULL	Fecha del pago/emisión
periodicidad	Enum	NULL	MENSUAL, TRIMESTRAL, SEMESTRAL, ANUAL, UNICO
fechaVencimiento	Long (timestamp)	NULL	Fecha de próximo vencimiento
notas	String	NULL	Observaciones

Entidad: Recordatorio

Atributo	Tipo	Restricciones	Descripción
id	Long	PK, autoincremental	Identificador único
vehiculoid	Long	FK → Vehiculo, NOT NULL	Vehículo asociado
concepto	String	NOT NULL	Descripción del recordatorio
fechaLimite	Long (timestamp)	NULL	Fecha límite para la acción
kilometrajeLimite	Double	NULL	Kilometraje límite (si aplica)
completado	Boolean	NOT NULL, default false	Si se ha marcado como realizado
fechaCompletado	Long (timestamp)	NULL	Fecha en la que se completó
notas	String	NULL	Observaciones adicionales

4.1.2. Relaciones entre entidades

Las relaciones entre las entidades del modelo de datos son las siguientes:

Entidad origen	Relación	Entidad destino	Cardinalidad	Política de borrado
Vehiculo	tiene	Repostaje	1:N	CASCADE
Vehiculo	tiene	Mantenimiento	1:N	CASCADE
Vehiculo	tiene	GastoPeriodico	1:N	CASCADE
Vehiculo	tiene	Recordatorio	1:N	CASCADE

Todas las relaciones parten de la entidad Vehiculo como entidad principal. Cuando se elimina un vehículo, todos los registros asociados se eliminan automáticamente gracias a la política de eliminación en cascada (ON DELETE CASCADE), garantizando la integridad referencial de la base de datos.

4.2. Esquema de la base de datos (Room / SQLite)

A continuación se presenta la definición conceptual de las tablas que compondrán la base de datos SQLite gestionada a través de Room. Cada tabla se corresponde con una entidad del modelo descrito anteriormente y será implementada como una clase @Entity en Kotlin.

La base de datos se denominará **gestcar_database** y se definirá mediante una clase abstracta que extienda de RoomDatabase, incluyendo los DAOs (Data Access Objects) necesarios para cada entidad. Room se encargará de generar automáticamente las consultas SQL y de gestionar las migraciones del esquema.

Los campos de tipo fecha se almacenan como Long (timestamp en milisegundos desde epoch Unix), utilizando TypeConverters de Room para la conversión entre Long y objetos Date o LocalDateTime de Kotlin. Los campos enum se almacenarán como String mediante un TypeConverter que invoque .name para la serialización y valueOf() para la deserialización.

5. Diseño de la arquitectura

5.1. Patrón arquitectónico: MVVM

GestCar seguirá el patrón de arquitectura MVVM (Model–View–ViewModel), recomendado por Google como la arquitectura de referencia para el desarrollo de aplicaciones Android modernas.

5.1.1. Capa Model (Modelo)

La capa de modelo encapsula la lógica de acceso a datos y las entidades del dominio. En GestCar, esta capa se compone de los siguientes elementos:

- **Entities (@Entity):** Clases de datos Kotlin anotadas con @Entity que representan las tablas de la base de datos. Cada entidad define los campos, claves primarias, claves foráneas e índices.
- **DAOs (@Dao):** Interfaces anotadas con @Dao que definen las operaciones de acceso a datos (consultas SQL) sobre cada entidad. Room genera la implementación automáticamente.
- **Database (@Database):** Clase abstracta que extiende RoomDatabase y declara las entidades y los DAOs. Se instancia como Singleton para evitar múltiples conexiones.
- **Repositorios (Repository):** Clases que actúan como intermediarias entre los DAOs y los ViewModels, abstrayendo la fuente de datos y permitiendo añadir lógica de negocio o caché si fuera necesario en el futuro.

5.1.2. Capa View (Vista)

La capa de vista se encarga de la presentación de la interfaz de usuario y de la gestión de las interacciones del usuario. En GestCar se implementará utilizando Jetpack Compose, el framework declarativo de UI de Android. Los componentes principales son:

- **Screens (Composables):** Funciones @Composable que definen cada pantalla de la aplicación (listado de vehículos, formulario de repostaje, estadísticas, etc.).
- **Navigation:** Se utilizará Jetpack Navigation Compose para gestionar la navegación entre pantallas mediante un NavHost y rutas definidas como sealed class o constantes.
- **Components reutilizables:** Composables genéricos como tarjetas de vehículo, campos de formulario personalizados, indicadores de estado de recordatorios, etc.

5.1.3. Capa ViewModel

Los ViewModels actúan como puente entre el modelo y la vista, gestionando el estado de la UI y la lógica de presentación. Cada pantalla principal tendrá su propio ViewModel que:

- Expone el estado de la UI mediante StateFlow o State (Compose), que la vista observa de forma reactiva.
- Recibe eventos del usuario desde la vista y los transforma en operaciones sobre el repositorio.
- Gestiona la lógica de validación de formularios antes de persistir los datos.
- Ejecuta operaciones de base de datos en coroutines (viewModelScope) para no bloquear el hilo principal.
- Calcula estadísticas y métricas derivadas (consumo medio, coste/km) a partir de los datos del repositorio.

5.2. Diagrama de componentes

A continuación se describe la estructura de paquetes y componentes principales de la aplicación, organizados según la arquitectura MVVM:

Paquete	Componente	Responsabilidad
com.gestcar.data.entity	Vehiculo, Repostaje, Mantenimiento, GastoPeriodico, Recordatorio	Definición de entidades Room con anotaciones @Entity
com.gestcar.data.dao	VehiculoDao, RepostajeDao, MantenimientoDao, GastoPeriodicoDao, RecordatorioDao	Interfaces DAO con operaciones CRUD y consultas personalizadas
com.gestcar.data.database	GestCarDatabase	Clase @Database Singleton que configura Room y provee acceso a los DAOs
com.gestcar.data.converter	DateConverter, EnumConverter	TypeConverters para convertir tipos complejos (fechas, enums) a tipos SQLite
com.gestcar.data.repository	VehiculoRepository, RepostajeRepository, MantenimientoRepository, GastoRepository, RecordatorioRepository	Repositorios que encapsulan la lógica de acceso a datos
com.gestcar.ui.screens	VehiculoListScreen, VehiculoDetailScreen, RepostajeFormScreen, MantenimientoFormScreen, EstadisticasScreen, RecordatorioScreen, ...	Composables que definen las pantallas de la app
com.gestcar.ui.components	VehiculoCard, FormField, StatCard, ReminderBadge, ...	Composables reutilizables para elementos comunes de UI
com.gestcar.ui.navigation	NavGraph, Screen (sealed class)	Configuración de navegación y definición de rutas
com.gestcar.ui.viewmodel	VehiculoViewModel, RepostajeViewModel, MantenimientoViewModel, EstadisticasViewModel, RecordatorioViewModel	ViewModels con lógica de presentación y gestión de estado
com.gestcar.ui.theme	Theme, Color, Type	Definición del tema Material 3 (colores, tipografía, formas)
com.gestcar.util	DateUtils, CalculationUtils, CsvExporter	Utilidades compartidas (formateo de fechas, cálculos, exportación)

5.3. Flujo de datos

El flujo de datos en la aplicación sigue un patrón unidireccional (UDF – Unidirectional Data Flow), característico de las arquitecturas MVVM modernas con Compose:

- **Evento del usuario → Vista → ViewModel:** El usuario interactúa con la interfaz (pulsar un botón, rellenar un campo). La vista captura el evento y lo envía al ViewModel correspondiente mediante una función pública.
- **ViewModel → Repository → DAO → Room/SQLite:** El ViewModel procesa el evento, aplica validaciones y, si procede, invoca al repositorio. El repositorio delega en el DAO, que ejecuta la operación sobre la base de datos Room.
- **Room/SQLite → DAO → Repository → ViewModel → Vista:** Los cambios en la base de datos se propagan automáticamente a través de Flow (Kotlin coroutines). El ViewModel actualiza su StateFlow, y la vista recompone automáticamente los elementos afectados.

Este flujo garantiza que la vista siempre refleje el estado más reciente de los datos, sin necesidad de refrescos manuales.

5.4. Gestión de dependencias

Para la inyección de dependencias se evaluará el uso de Hilt (basado en Dagger), que permite inyectar automáticamente los repositorios en los ViewModels y la base de datos en los repositorios, reduciendo el acoplamiento entre componentes. Alternativamente, si la complejidad del proyecto no justifica Hilt, se utilizará un patrón de Service Locator manual con un objeto Kotlin que actúe como contenedor de dependencias.

6. Diseño de la interfaz de usuario

6.1. Principios de diseño

La interfaz de usuario de GestCar se diseñará siguiendo los principios de Material Design 3 (Material You), el sistema de diseño de Google para aplicaciones Android. Los principios fundamentales que guiarán el diseño son:

- **Sencillez:** Interfaces limpias con la información justa y necesaria. Se evitarán pantallas sobrecargadas de elementos.
- **Consistencia:** Uso uniforme de componentes, colores, tipografía e iconografía en toda la aplicación.
- **Accesibilidad:** Tamaños de fuente legibles, contraste adecuado, áreas táctiles de al menos 48dp y soporte para contentDescription.
- **Retroalimentación inmediata:** Mensajes de confirmación, errores y estados de carga claramente visibles para el usuario.
- **Navegación intuitiva:** Uso de una barra de navegación inferior (NavigationBar) para las secciones principales y navegación jerárquica con botón de retroceso.

6.2. Mapa de navegación

La estructura de navegación de GestCar se organiza en torno a una barra de navegación inferior con las siguientes secciones principales:

Sección	Icono sugerido	Pantallas accesibles
Vehículos	DirectionsCar	Listado de vehículos → Detalle de vehículo → Formulario alta/edición
Gastos	Payments	Resumen de gastos del vehículo activo → Detalle por categoría → Formulario alta/edición de gasto periódico
Repostajes	LocalGasStation	Historial de repostajes → Formulario nuevo repostaje → Detalle de repostaje
Mantenimiento	Build	Historial de operaciones → Formulario nueva operación → Detalle de operación
Más	MoreHoriz	Recordatorios → Estadísticas → Exportar datos → Ajustes

6.3. Descripción de pantallas principales

6.3.1. Pantalla principal – Listado de vehículos

Es la pantalla de inicio de la aplicación. Muestra una lista vertical de tarjetas (Cards) con la información resumida de cada vehículo registrado. Cada tarjeta incluye la imagen del vehículo (o un icono genérico según el tipo), la marca y modelo, la matrícula, el kilometraje actual y el número de recordatorios pendientes (si los hay). En la esquina superior derecha se muestra un indicador con el número de alertas activas. Un botón flotante (FAB) en la esquina inferior derecha permite añadir un nuevo vehículo. Si no hay vehículos registrados, se muestra un estado vacío (empty state) con un mensaje y un botón de acción.

6.3.2. Pantalla de detalle de vehículo

Muestra toda la información de un vehículo concreto organizada en secciones colapsables o pestañas. La cabecera incluye la imagen del vehículo, marca, modelo, año y matrícula. Se presentan indicadores rápidos (chips) con el kilometraje actual, el consumo medio y el coste por kilómetro. Debajo, se organizan las secciones de últimos repostajes, últimos mantenimientos, gastos periódicos activos y recordatorios pendientes. Cada sección muestra los 3-5 registros más recientes con un enlace «Ver todos». Los botones de editar y eliminar vehículo se ubican en el menú de acción (tres puntos) de la barra superior.

6.3.3. Formularios de registro

Todos los formularios de la aplicación (alta de vehículo, repostaje, mantenimiento, gasto periódico y recordatorio) siguen una estructura común: una barra superior con el título de la operación y botón de cierre/retroceso, campos de texto agrupados lógicamente con etiquetas descriptivas, selectores de fecha (DatePicker) para los campos de fecha, menús desplegables (ExposedDropdownMenu) para campos con opciones predefinidas (tipo de vehículo, tipo de combustible, tipo de mantenimiento, periodicidad), validación en línea que muestra errores debajo de cada campo inválido, y un botón principal de «Guardar» en la parte inferior.

6.3.4. Pantalla de estadísticas

Presenta las métricas calculadas del vehículo seleccionado en un diseño basado en tarjetas de estadísticas (StatCards). La parte superior muestra los indicadores clave: consumo medio (L/100km), coste total acumulado, coste por kilómetro y número total de operaciones. Debajo, un gráfico de barras muestra el desglose de gastos por categoría (combustible, mantenimiento, reparaciones, gastos periódicos). Un segundo gráfico de líneas muestra la evolución mensual de gastos. Se incluye un selector de período temporal y un botón de exportación a CSV.

6.3.5. Pantalla de recordatorios

Lista los recordatorios del vehículo activo organizados por estado: vencidos (indicador rojo), próximos a vencer (indicador ámbar, menos de 30 días o 1000 km) y pendientes (indicador verde). Cada elemento muestra el concepto, la fecha límite, el kilometraje límite (si aplica) y botones de acción rápida para marcar como completado, editar o eliminar. Un FAB permite crear nuevos recordatorios.

6.4. Paleta de colores propuesta

La paleta de colores seguirá el sistema dinámico de Material Design 3 (Material You), adaptándose automáticamente al wallpaper del dispositivo en Android 12+. No obstante, se define una paleta base para dispositivos anteriores o como fallback:

Elemento	Color	Código hex	Uso
Primary	Azul oscuro	#1F4E79	Botón principal, barra superior, acentos
Secondary	Azul claro	#4A90D9	Elementos secundarios, iconos activos
Tertiary	Verde azulado	#2E8B57	Indicadores positivos, consumo bajo
Error	Rojo	#B3261E	Alertas, recordatorios vencidos, errores
Warning	Ámbar	#F9A825	Recordatorios próximos, advertencias
Background	Blanco/Gris claro	#FAFAFA	Fondo de pantallas
Surface	Blanco	#FFFFFF	Tarjetas y superficies elevadas
On Primary	Blanco	#FFFFFF	Texto sobre colores primarios
On Surface	Gris oscuro	#1C1B1F	Texto principal sobre superficies

7. Tecnologías y herramientas

7.1. Justificación del stack tecnológico

A continuación se detallan las tecnologías seleccionadas para el desarrollo de GestCar, junto con la justificación de cada elección:

Tecnología	Versión prevista	Justificación
Kotlin	1.9.x	Lenguaje oficial para desarrollo Android recomendado por Google. Ofrece null safety, coroutines para asincronía, data classes para entidades y extensiones que simplifican el código.
Android Studio	Ladybug+	IDE oficial de Google para Android. Incluye herramientas de diseño, depuración, emuladores y soporte nativo para Compose Preview.
Jetpack Compose	1.6.x	Framework declarativo de UI que sustituye al sistema de vistas XML. Permite construir interfaces reactivas con menos código y mejor testabilidad.
Room	2.6.x	Biblioteca de persistencia sobre SQLite que proporciona verificación de consultas SQL en tiempo de compilación, migraciones automáticas y soporte nativo para Flow/coroutines.
SQLite	3.x (integrada)	Base de datos relacional integrada en Android. Ligera, sin necesidad de servidor externo, ideal para almacenamiento local en dispositivos móviles.
Navigation Compose	2.7.x	Componente de Jetpack para gestión de la navegación entre composables. Soporta argumentos tipados, deep links y animaciones de transición.
Material Design 3	1.2.x	Sistema de diseño de Google que proporciona componentes visuales predefinidos, tematización dinámica y accesibilidad integrada.
Kotlin Coroutines	1.8.x	Framework de programación asincrónica nativo de Kotlin. Permite ejecutar operaciones de base de datos y cálculos sin bloquear la interfaz.
Git / GitHub	Última	Sistema de control de versiones distribuido. Permite llevar un historial completo de cambios, facilita la reversión de errores y sirve como copia de seguridad remota del código.
Gradle (Kotlin DSL)	8.x	Sistema de construcción (build system) del proyecto. La variante Kotlin DSL ofrece autocompletado y verificación de tipos en los archivos de configuración.

7.2. Dependencias principales (build.gradle.kts)

Las principales dependencias que se incluirán en el archivo build.gradle.kts del módulo app son las siguientes (las versiones podrán actualizarse durante el desarrollo):

- **androidx.core:core-ktx** – Extensiones de Kotlin para APIs de Android.
- **androidx.lifecycle:lifecycle-runtime-ktx** – Gestión del ciclo de vida con coroutines.
- **androidx.lifecycle:lifecycle-viewmodel-compose** – Integración de ViewModel con Compose.
- **androidx.activity:activity-compose** – Integración de Activity con Compose.
- **androidx.compose.ui:ui** – Framework de UI Compose.
- **androidx.compose.material3:material3** – Componentes Material Design 3.
- **androidx.navigation:navigation-compose** – Navegación entre composables.
- **androidx.room:room-runtime** y **room-ktx** – Persistencia Room con soporte para coroutines.
- **androidx.room:room-compiler** (kapt/ksp) – Procesador de anotaciones de Room.
- **org.jetbrains.kotlinx:kotlinx-coroutines-android** – Soporte de coroutines para Android.

7.3. Entorno de desarrollo y pruebas

El desarrollo se realizará en un entorno compuesto por:

- **Sistema operativo:** Windows 11 / macOS (compatible con ambos).
- **IDE:** Android Studio (versión Ladybug o posterior) con los plugins de Kotlin y Compose actualizados.
- **Emulador:** Pixel 7 API 34 como dispositivo virtual principal de pruebas. Se probará también en un dispositivo físico si está disponible.
- **Control de versiones:** Repositorio privado en GitHub con commits frecuentes y mensajes descriptivos siguiendo la convención Conventional Commits.
- **Pruebas:** Se contemplan pruebas unitarias de los ViewModels y Repositorios con JUnit 5, y pruebas instrumentadas básicas de la base de datos con AndroidX Test.

8. Planificación preliminar

A continuación se presenta una planificación orientativa del desarrollo del proyecto, organizada en fases con sus actividades y duraciones estimadas:

Fase	Actividades	Duración estimada	Entregable
Fase 1: Análisis y diseño	Análisis de requisitos, casos de uso, modelo de datos, diseño de arquitectura, diseño de interfaz	3 semanas	Documento de análisis y diseño (este documento)
Fase 2: Configuración del proyecto	Creación del proyecto Android Studio, configuración de dependencias, estructura de paquetes, repositorio Git	1 semana	Proyecto base funcional
Fase 3: Implementación del modelo	Entidades Room, DAOs, base de datos, TypeConverters, repositorios	2 semanas	Capa de datos completa y testada
Fase 4: Implementación de la UI	Pantallas Compose, navegación, formularios, componentes reutilizables, tema Material 3	3 semanas	Interfaz de usuario completa
Fase 5: Implementación de ViewModels	ViewModels, lógica de presentación, validaciones, cálculos automáticos, conexión con repositorios	2 semanas	Lógica de negocio integrada
Fase 6: Estadísticas y extras	Pantalla de estadísticas, gráficos, exportación CSV, recordatorios con alertas	2 semanas	Funcionalidades avanzadas
Fase 7: Pruebas y corrección	Pruebas unitarias, pruebas instrumentadas, corrección de errores, optimización	1 semana	Aplicación estable y testada
Fase 8: Documentación final	Redacción de la memoria técnica, manual de usuario, preparación de la defensa	2 semanas	Memoria completa y presentación

La duración total estimada del proyecto es de aproximadamente 16 semanas. Las fases pueden solaparse parcialmente, especialmente las fases 4 y 5, que se desarrollarán de forma iterativa. Dichos tiempos podrían variar resultando en el aumento o reducción del tiempo de desarrollo del proyecto.

9. Glosario de términos

Término	Definición
API	Application Programming Interface. Conjunto de definiciones y protocolos para integrar componentes de software.
APK	Android Package Kit. Formato de archivo utilizado para distribuir e instalar aplicaciones en Android.
Composable	Función anotada con @Composable en Jetpack Compose que define un elemento de interfaz de usuario.
Coroutine	Patrón de concurrencia en Kotlin que permite ejecutar tareas asincrónicas de forma secuencial y sin bloquear hilos.
CRUD	Create, Read, Update, Delete. Las cuatro operaciones básicas sobre datos persistentes.
DAO	Data Access Object. Patrón de diseño que abstrae las operaciones de acceso a la base de datos.
DXA	Unit of measurement in OOXML. 1 DXA = 1/20 punto, 1440 DXA = 1 pulgada.
Flow	API reactiva de Kotlin para emitir múltiples valores de forma asincrónica a lo largo del tiempo.
ITV	Inspección Técnica de Vehículos. Revisión periódica obligatoria en España.
MVVM	Model–View–ViewModel. Patrón arquitectónico que separa la lógica de negocio de la interfaz de usuario.
Room	Biblioteca de persistencia de Android Jetpack que proporciona una capa de abstracción sobre SQLite.
SQLite	Motor de base de datos relacional ligero integrado en el sistema operativo Android.
StateFlow	Implementación de Flow en Kotlin que mantiene el último valor emitido y notifica a los observadores de cambios.
UDF	Unidirectional Data Flow. Patrón de arquitectura donde los datos fluyen en una única dirección.
ViewModel	Componente de Android Jetpack diseñado para almacenar y gestionar datos relacionados con la UI de forma consciente del ciclo de vida.

10. Conclusiones de la fase de análisis y diseño

El presente documento recoge el trabajo de análisis y diseño realizado durante la primera fase del proyecto GestCar. A lo largo de las secciones anteriores se han definido con detalle los requisitos funcionales y no funcionales del sistema, se han identificado y descrito los casos de uso principales, se ha diseñado un modelo de datos relacional que cubre todas las necesidades de persistencia de la aplicación y se ha definido una arquitectura MVVM sólida y escalable.

El diseño de la interfaz de usuario se ha planteado siguiendo los estándares de Material Design 3, garantizando una experiencia coherente, accesible e intuitiva para el usuario final. La selección del stack tecnológico (Kotlin, Jetpack Compose, Room, Navigation Compose) responde a las recomendaciones oficiales de Google para el desarrollo Android moderno y asegura compatibilidad a largo plazo con el ecosistema Android.

La planificación preliminar establece un calendario realista de 16 semanas que permite abordar el desarrollo de forma iterativa e incremental, con margen para pruebas y documentación final.

Con la finalización de esta primera entrega, el proyecto cuenta con una base sólida para iniciar la fase de implementación, disponiendo de un mapa claro de qué construir, cómo estructurarlo y qué tecnologías emplear.